

Long Polling

Short polling vs long polling — with the "how it holds" trick explained

1. What they are (one line each)

- **Short Polling** — Client asks the server "anything new?" every X seconds. Server responds immediately, even if there's nothing.
- **Long Polling** — Client asks once, and the server **holds the request open** until something happens (or timeout), then responds. Client immediately re-asks.

2. Short polling — React example

Backend (Express):

```
// server.js
app.get('/messages', (req, res) => {
  // Just return whatever is currently there – instantly
  res.json({ messages: store.getAll() });
});
```

Frontend (React):

```
function ChatShort() {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const interval = setInterval(async () => {
      const res = await fetch('/messages');
      const data = await res.json();
      setMessages(data.messages);
    }, 3000); // ← ask every 3 seconds

    return () => clearInterval(interval);
  }, []);

  return <ul>{messages.map(m => <li
    key={m.id}>{m.text}</li>)}</ul>;
}
```

Flow:

```
Client -> "anything new?" -> Server -> "no" (immediate)
[wait 3s]
Client -> "anything new?" -> Server -> "no" (immediate)
[wait 3s]
Client -> "anything new?" -> Server -> "yes, here"
```

Wasteful — most requests return nothing.

3. Long polling — simplest React example

Frontend (React) — that's the whole client:

```
import { useEffect, useState } from 'react';

function App() {
  const [message, setMessage] = useState('Waiting for news...');

  useEffect(() => {
    async function poll() {
      const res = await fetch('http://localhost:3000/news'); // hangs until reply
      const data = await res.json();
      setMessage(data.message);
      poll(); // ask again
    }
    poll();
  }, []);

  return <h1>{message}</h1>;
}

export default App;
```

Just 3 important lines:

1. `await fetch(...)` — sends the request and waits.
2. `setMessage(data.message)` — show the reply when it arrives.
3. `poll()` — call itself again to wait for the next message.

Backend (Express) — to test it:

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());

let waiting = null;

app.get('/news', (req, res) => {
  waiting = res; // hold the response, don't reply yet
});

app.get('/send/:msg', (req, res) => {
  if (waiting) {
    waiting.json({ message: req.params.msg });
    waiting = null;
  }
  res.send('sent');
});

app.listen(3000);
```

How to see it work:

1. Run server: `node server.js`
2. Run React app — it shows: **"Waiting for news..."**

3. Open new tab: `http://localhost:3000/send/Hello`
4. App instantly updates to **"Hello"**
5. App is now waiting again — send another: `http://localhost:3000/send/world`
6. App updates to **"World"**

4. How is it "holding"? (the part that confuses everyone)

■ The whole trick

The server just doesn't call `res.json()` yet. It saves the `res` object in a variable and walks away. The connection stays open as long as you haven't called `res.json()`, `res.send()`, or `res.end()`. There's no special API — just a deferred reply.

Normal endpoint (replies instantly):

```
app.get('/news', (req, res) => {  
  res.json({ message: 'hi' }); // replies immediately  
});
```

Long polling endpoint (defers the reply):

```
let waiting = null;  
  
app.get('/news', (req, res) => {  
  waiting = res; // ← save the response object in a variable  
  // (no res.json here!)  
  // Express function ends, but the connection STAYS OPEN  
});
```

That's it. The browser's `fetch()` is still waiting for a reply that hasn't come yet.

How does it finally reply?

Some **other** code path (a different request, an event, a timer) calls `waiting.json(...)` later:

```
app.get('/send/:msg', (req, res) => {  
  if (waiting) {  
    waiting.json({ message: req.params.msg }); // ← held connection finally replies  
    waiting = null;  
  }  
  res.send('sent');  
});
```

When `/send/Hello` is hit, this code calls `waiting.json(...)` — sending a reply on the held connection from before. The browser's hanging `fetch()` finally resolves.

5. Visual timeline

TIME CLIENT (React) SERVER

```
t=0 fetch('/news') ■■■■■■■■■■ app.get('/news') runs:
waiting = res
(function ends – but connection still open!)

t=0.. (still waiting...) (waiting variable holds the res)
(still waiting...)
(still waiting...)

t=15s Someone hits /send>Hello:
waiting.json({ message: 'Hello' })
(reply finally sent on held connection)

t=15s ■■■■ reply arrives ■■■■■■■■■■
await fetch resolves
setMessage('Hello')
poll() called again

t=15s fetch('/news') ■■■■■■■■■■ waiting = res (new one)
(waits again...)
```

The "hold" is just **JavaScript saving the res object** and not calling `.json()` on it yet. The browser keeps the TCP connection open because it's waiting for the HTTP response to finish.

6. A clearer example — auto-reply after 10 seconds

To make it crystal clear, here's a version where the server holds for **exactly 10 seconds**, then replies:

```
app.get('/news', (req, res) => {
  console.log('Got request, holding for 10 seconds...');

  setTimeout(() => {
    res.json({ message: 'Hello after 10s!' }); // NOW reply
  }, 10000);
});
```

What happens:

1. React calls `fetch('/news')` — request goes to server.
2. Server logs "Got request, holding for 10 seconds..."
3. Server schedules a reply in 10 seconds via `setTimeout`.
4. The Express function returns, but **res is still alive** (held by the closure inside `setTimeout`).
5. The browser sees the request still pending in DevTools Network tab.
6. After 10 seconds, `setTimeout` fires → `res.json(...)` → reply is sent.
7. React's `await fetch` resolves → `setMessage` runs → UI updates.

■ See it yourself

Open DevTools → Network tab → click on the `/news` request → status shows **"Pending"** for 10 seconds, then completes. That's the held connection visualized.

7. The mental model

A regular request:

```
fetch ■■■ server replies ■■■ done  
(a few ms)
```

A long polling request:

```
fetch ■■■ server saves res, doesn't reply ■■■ .....waiting..... ■■■ something  
triggers  
res.json() ■■■ done  
(could be 1s or 5 min)
```

The "holding" is literally **the server choosing to delay calling `res.json()`**. There's no special API for it. Express doesn't know it's "long polling" — it just sees a handler that hasn't sent a response yet, so it keeps the connection alive.

8. Two things that can trigger the reply

1. **An event** — like in our chat example, when someone calls `/send/Hello`, that handler reaches into `waiting` and calls `.json()` on it.
2. **A timeout** — the server replies after `N` seconds even if nothing happened, so the connection doesn't stay open forever.

Production long polling combines both:

```
app.get('/news', (req, res) => {  
  waiting = res;  
  
  // safety net – if nothing happens in 30s, reply with empty  
  const timer = setTimeout(() => {  
    res.json({ message: null });  
    waiting = null;  
  }, 30000);  
  
  // if client disconnects, clear the timer  
  res.on('close', () => clearTimeout(timer));  
});
```

9. Short polling vs Long polling

Aspect	Short Polling	Long Polling
Client behavior	Asks every X seconds	Asks once, waits for response, re-asks
Server behavior	Replies instantly	Holds connection open until data or timeout
Latency	Up to X seconds delay	Near-instant (whenever event happens)
Network requests	Many empty ones	Few, mostly meaningful
Server resources	Low per request, high total	Each request holds a connection (more memory)
Complexity	Trivial	Need queue + timeout + cleanup
Use when	Updates rare, latency unimportant	Real-time feel needed, but WebSocket overkill

When to actually use each

- **Short polling**: dashboards refreshing every minute, status checks, simple use cases.
- **Long polling**: chat fallbacks, notifications, when you need real-time but can't use WebSockets/SSE (e.g. behind strict corporate proxies).
- **Better alternatives in 2026: Server-Sent Events (SSE)** for one-way push, **WebSockets** for two-way real-time. Long polling is mostly a fallback now.

10. Interview Q&A

■ How is long polling different from short polling?

Short polling — client repeatedly asks the server every X seconds; server always replies immediately.

Long polling — client asks once, server holds the request open until data is available (or a timeout fires), then replies. Client immediately re-asks. Long polling has lower latency and fewer wasted requests, but holds connections.

■ How does the server actually "hold" a long polling request?

It just doesn't call `res.json()` or `res.end()` yet. Express keeps the connection open as long as the response hasn't been finalized. The handler stores the `res` object in a variable and returns; Express never closes the connection until something later calls `.json()` or `.send()` on that `res` object.

■ What triggers the held response to finally reply?

Two things: (1) an event — typically another HTTP request or an in-process event that knows about the held `res`, calls `.json()` on it, and finalizes the response. (2) A safety timeout — usually 30 seconds — that replies with empty data so the connection doesn't hang forever.

■ What are the downsides of long polling?

Each open request holds a connection (RAM + connection limit per server). Doesn't scale as well as WebSockets/SSE for thousands of concurrent users. Reconnection logic is needed on the client. Doesn't work efficiently across horizontally-scaled backends without a shared coordination layer (Redis pub/sub, etc).

■ What's better than long polling in 2026?

Server-Sent Events (SSE) — one-way push from server to client, simpler than WebSockets, works over standard HTTP. **WebSockets** — full duplex, lowest latency, ideal for chat and games. Long polling remains as a fallback when SSE/WebSockets are blocked by proxies or older infrastructure.

■ How would you handle long polling for thousands of users?

Use a pub/sub layer (Redis, NATS) so any backend instance can wake up any held request when an event occurs. Set a max wait time (e.g. 30s) and have clients auto-reconnect. Monitor concurrent connections — at scale, switch to SSE or WebSockets which are designed for this.

11. TL;DR

■ The whole story in 4 lines

1. Client calls `fetch('/news')`.
2. Server saves the `res` object in a variable and DOESN'T call `res.json()` yet. The connection stays open.
3. Later, when something interesting happens, some other code calls `waiting.json(...)` — finalizing the held response.
4. Client's `await fetch` finally resolves; client immediately re-asks. Repeat forever.

One-line interview answer

■ If you only remember one sentence...

"Long polling "holds" a request by deferring `res.json()` — the server saves the response object in a variable and only finalizes the reply when something interesting happens (or a timeout fires). The client's `fetch` hangs until then, and immediately re-asks once it resolves."

Reference for interview prep — Akshay Chavhan